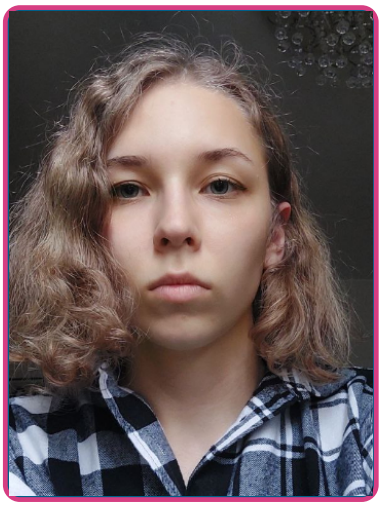
Системный программист

Земляная Дарья

+7(909) 155 94 99

darijazeml@inbox.ru

github.com/Dariazeml1007



Основные навыки:
C • C++ • x86/x64 ASM • Python • Ruby

Инструменты:
Git • CMake • GDB • Valgrind • Linux

Языки:
Русский (родной) • Английский (B1)

Образование

Московский Физико-Технический Институт *Москва, Россия* — 2024 - Настоящее время

- Факультет радиотехники и кибернетики (ФРКТ)
- Прикладная математика и физика

Лицей "Вторая школа" *Москва, Россия* — 2021 - 2024

- Мат - инфо профиль
- Призер олимпиад по математике и физике

Спецкурсы

- **Компиляторные технологии и промышленное программирование на C** (Дединский И.Р)
- **Введение в промышленное программирование на C++** (Владимиров К.И)
- **Введение в архитектуру вычислительных систем** (Sber)
- **Введение в тензорные компиляторы** (Sber)
- **Тестовые генераторы**

Опыт работы

Стажер по анализу данных *MTC Digital* — 06/2024 - 08/2024

Стажировка по Big Data

- Получила практический опыт работы с PySpark для обработки больших данных
- Работала с системой контроля версий Git в среде GitLab
- Участвовала в подготовке и анализе данных для проектов AI/ML
- Развила понимание практического применения моделей AI в бизнесе
- Участвовала в командных встречах и обсуждениях проектов

Tech Stack PySpark, Git/GitLab, Python, Big Data concepts

Ключевые учебные проекты @ Huawei/МФТИ 1 курс

Под руководством Ильи Рудольфовича Дединского | Выполнено 10+ проектов по системному программированию

Эмулятор процессора *Системное программирование* — 2024

- **Реализация на чистом C**: Ассемблерный компилятор + стековая ВМ
- **18 инструкций**:
  - Арифметика: ADD/SUB/MUL/DIV
  - Операции со стеком: PUSH/POP

  - Управление потоком: JMP/CALL/RET
- Особенности: поддержка меток, примеры рекурсии (факториал)
- [Ссылка на github]

Tech Stack C, Синтаксис ассемблера

Низкоуровневая реализация printf *Интеграция ассемблера и C* **2025**

- **Интерфейс C-ASM** через extern вызовы
- **Поддержка форматов: %d %s %c + числовые (%b/%o/%x)**
- Унифицированная обработка чисел для форматов с основанием
- [Ссылка на github]

Tech Stack Nasm, System V ABI

Оптимизатор множества Мандельброта *SIMD программирование* **2025**

- **3 реализации**:
  - Базовый скалярный вариант
  - Автовекторизованный (-O3)
  - Ручные AVX2 инструкции (m256)
- Интерактивная визуализация с TxLib
- [Ссылка на github]

Tech Stack C, AVX2 Intrinsics, TxLib

Оптимизация хэш-таблицы для строк *C, AVX2, ASM, Valgrind* **2025**

- **Методичная оптимизация**: hash() → strcmp() → search() через CRC32/AVX2 интринсики
- **Профилирование**: Callgrind для анализа тактов, clock_gettime для end-to-end замеров
- **Результат**: **1.76× ускорение** поиска при 8 asm-инструкциях; коэффициент улучшения = 220
- **Инсайт**: интринсики + компиляторные оптимизации (-O3) эффективнее ручного asm в контексте
- [Ссылка на github]

Tech Stack C, x86-64 ASM, AVX2 Intrinsics, Valgrind/Callgrind, System V ABI

Введение в промышленное программирование на C++”

Под руководством Владимирова Константина Игоревича

2Q Cache Simulator *C++17, STL* **2025**

- **Алгоритм**: реализация Two-Queue с доступом через std::unordered_map
- **Метаданные**: список A_out для отслеживания «вытесненных» элементов, точность 90% от ideal cache
- [Ссылка на github]

Tech Stack C++17, STL, CMake

BST с range queries *C++17, Алгоритмы* **2025**

- **Функционал**: вставка ключей, подсчёт элементов в диапазоне $[L; R)$, dump в Graphviz
- **Оптимизация**: балансировка + кэширование метаданных для ускорения range-запросов
- [Ссылка на github]

Tech Stack C++17, AVL Trees, CMake, Benchmarking, GoogleTest

Язык ParaCL *C++17, Flex/Bison* **2026**

- **Toolchain**: лексический анализ (Flex) → парсинг (Bison) → AST → интерпретация
- **Архитектура**: RAII для управления памятью AST, scoped variables, arithmetic evaluation
- **Качество**: модульные тесты на GoogleTest, CMake-сборка под Linux/WSL
- [Ссылка на github]

Tech Stack C++17, Flex, Bison, AST, GoogleTest

Проект по курсу "Средства симмуляции"

Эмулятор CPU с DSL-ассемблером *C++, Ruby* **2025**

- **ISA**: 14 инструкций, mixed encoding (RISC-style + custom), поддержка прерываний
- **Toolchain**: Ruby-based micro-assembler → бинарный образ → эмуляция в C++
- **Дизайн**: разделение frontend (DSL) и backend (emulator) для тестируемости
- [Ссылка на github]

Tech Stack C++17, Ruby, DSL, Emulation, Custom ISA